

EULER METHODS FOR THE LOTKA-VOLTERRA MODEL: FORWARD, BACKWARD, SYMPLECTIC

VIDUSHI MITTAL
UNIVERSITY OF CALIFORNIA, DAVIS
FINAL PROJECT, MATH 226C: NUMERICAL METHODS

June 11, 2026

1 Introduction to the Lotka–Volterra Model: Context

The Lotka-Volterra model is a pair of first-order nonlinear autonomous differential equations that capture the population dynamics of a predator-prey relationship. These equations receive their name from mathematicians Alfred James Lotka and Vito Volterra, who coincidentally found these equations around the same time, independently of each other, in 1925 and 1926 respectively.

Though created with the intention of representing predator-prey population dynamics, they can be used in applications to various other scenarios, including epidemiology, macroeconomics, and corporate competition.

2 Assumptions

In this context, we consider an environment with two species, a prey population and a predator population. We introduce the following assumptions before presenting our model:

1. The prey population has enough food for the entire population at any given time (i.e, the population growth of prey is not limited by food at any time)
2. The prey population is the only source of food for the predator population (i.e, the prey population determines the quantity of food supply for the predator population)
3. The rate of change of population is proportional to its size
4. Predators have an unlimited appetite.
5. The prey population increases exponentially when predators are not present, and predator population decreases exponentially when prey are not present.

3 Mathematical Formulation

To set up our model, we let $x(t), y(t) \geq 0$ denote the population density at time t of a prey and predator species, respectively. The general form for Lotka-Volterra is:

$$\begin{aligned}x' &= ax - bxy \\ y' &= -cy + dxy.\end{aligned}$$

Here, x represents a prey population, and y represents a predator population. The parameters a, b, c, d are positive and real-valued, and capture how the two species interact with each other. More specifically, a describes the prey's maximum growth rate (per capita), and b describes the effect of predators on the prey's death rate. Similarly, c captures the predator's death rate (per capita), and d encodes the predator's growth rate, dependent on the presence of prey. To get an idea of the correct behavior we are searching for in numerically computed solutions, we find equilibria, linearize via the Jacobian matrix, and classify stability.

Though the formulation of the model appears to be relatively simple, the system cannot be solved analytically in closed form. This is because the equations are nonlinear, and couple, meaning the rate of change of each variable

is dependent on a non linear combination of the others, so separation of variables to integrate is not feasible here.

Thus, for any practical application in the real world, we turn to numerical analysis to compute solutions. We will try to implement multiple Euler methods, Forward, Backward, and Symplectic, and compare the efficiency and accuracy.

4 Linearization via Jacobian Matrix

Setting the time derivatives equal to 0 yields two equilibria: $(0, 0)$ and $(\frac{c}{d}, \frac{a}{b})$. Contextually, the meaning of $(0, 0)$ is clear, at this point, both populations are extinct, and we consider the ecosystem dead. The other equilibrium point, $(\frac{c}{d}, \frac{a}{b})$ represents the population densities for both populations to coexist. At this point, the reproductive and mortality rates balance, and the populations fluctuate around this center.

We can briefly linearize the system to consider the stability of both equilibria. The Jacobian matrix is given by:

$$J(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix} = \begin{bmatrix} a - by & -bx \\ dy & dx - c \end{bmatrix}.$$

At the point $(0, 0)$,

$$J(0, 0) = \begin{bmatrix} a & 0 \\ 0 & -c \end{bmatrix}.$$

With one positive and one negative eigenvalue, this equilibrium is an unstable saddle. Biologically, this equilibrium is less relevant.

At the point $(\frac{c}{d}, \frac{a}{b})$,

$$J(\frac{c}{d}, \frac{a}{b}) = \begin{bmatrix} 0 & \frac{bc}{d} \\ \frac{ad}{b} & 0 \end{bmatrix}.$$

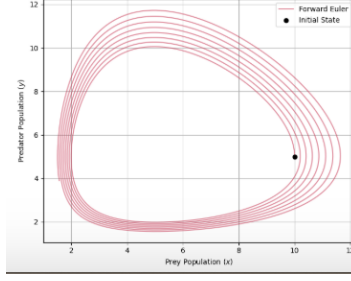
This results in eigenvalues $\lambda = \pm i\sqrt{ac}$. Since these satisfy $\text{Re}(\lambda) = 0$, the equilibrium point $(\frac{c}{d}, \frac{a}{b})$ is a neutral center. We will focus on behavior around this equilibrium point, now expecting that solutions in the neighborhood of this point should form closed periodic orbits.

5 Forward and Backward Euler Methods

5.1 Forward Euler In this context, using our parameters a, b, c, d , the scheme is written as

$$\begin{aligned} x_{n+1} &= x_n + h \cdot x_n(a - by_n) \\ y_{n+1} &= y_n + h \cdot y_n(-c + dx_n) \end{aligned}$$

While we will analyze this behavior in more depth in sections 9 and 10, we can see a quick implementation on a test case below:



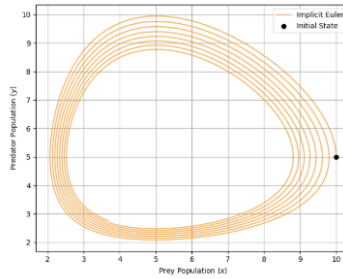
We see that solutions seem to be spiraling outward relative to the initial condition, an indicator this method is unstable and perhaps not the best choice.

5.2 Backward Euler In this context, using our parameters a, b, c, d , the scheme is written as

$$\begin{aligned} x_{n+1} &= x_n + h \cdot x_{n+1}(a - by_{n+1}) \\ y_{n+1} &= y_n + h \cdot y_{n+1}(-c + dx_{n+1}) \end{aligned}$$

In the code, to calculate the values x_{n+1}, y_{n+1} , we begin with an initial guess using a step of Forward Euler, then reframe the implicit equation as a root finding problem $G(u) = 0$, where $u = \begin{bmatrix} x_{n+1} \\ y_{n+1} \end{bmatrix}$. We use 10 iterations of Newton's method to solve this root-finding problem, utilizing the Jacobian matrix to solve $(I - hJ)\Delta u = -G$ for the next "piece" Δu that we add to our existing guess for u at each iteration.

We will revisit this in sections 9 and 10, but we again provide a quick implementation on a test case below:



We see that solutions seem to be spiraling inward relative to the initial condition, an indicator this method is likely stable, but still not the best choice.

6 Hamiltonian systems and Symplecticity

To begin this section, we introduce a definition. A Hamiltonian system of differential equations is a system such that there exists a function $H(x, y)$ that allows the system to be written as:

$$\begin{aligned} x' &= H_y(x, y) \\ y' &= -H_x(x, y). \end{aligned}$$

These systems have some useful properties. First, there exists a constant C so that $H(x, y) = C \quad \forall x, y$. This follows almost immediately from the definition. We can use the multivariate chain rule to write the rate of change

of $H(x(t), y(t))$ with respect to time t as:

$$\frac{dH}{dt} = H_x \frac{dx}{dt} + H_y \frac{dy}{dt}$$

. By definition, $\frac{dx}{dt} = H_y$, and $\frac{dy}{dt} = -H_x$. Replacing this back into the above equality, we are left with

$$\frac{dH}{dt} = H_x H_y - H_y H_x = 0, \quad (*)$$

leaving us to conclude $H(x(t), y(t))$ must be a constant function.

This gives some valuable insight into the behavior of the true solution, since it tells us that all solutions should be level curves of $H(x, y)$. This also means we can compute $H(x(0), y(0))$, and know that in the true solution,

$$H(x(0), y(0)) = H(x(t), y(t)) \quad \forall t.$$

We will refer to this as the invariant value of the Hamiltonian system. This property leads us to search for solvers that keep $|H(x(t), y(t)) - H(x(0), y(0))|$ small.

Furthermore, from (*), we can conclude

$$H_x dy + H_y dx = 0,$$

which tells us that the divergence is zero. This gives another property, that solutions are area-preserving. This means that, if we select a set of initial conditions $\{(x_0, y_0)\}$ such that these points form a boundary with area A , solve for the solutions for each initial condition, and then evaluate the solutions at some time t , the area formed by the points $\{(x(t), y(t))\}$ will still be equal to A . This is the definition of a symplectic system, for which symplectic solvers are much more suitable, since they satisfy this area preservation, and additionally bound $|H(x(t), y(t)) - H(x(0), y(0))|$.

7 Change of Variables to Attain Hamiltonian System

Lotka-Volterra is not inherently Hamiltonian as initially written. However, we can impose a clever change of variables to make it so. By defining,

$$\begin{aligned} u = \ln(x) &\implies x = e^u \\ v = \ln(y) &\implies y = e^v, \end{aligned}$$

we have a new system in terms of u and v :

$$\begin{aligned} u' &= a - be^v \\ v' &= -c + de^u. \end{aligned}$$

Let us briefly show this system is Hamiltonian. Say there existed a function $H(u, v)$ so that

$$\begin{aligned} u' &= H_v(u, v) = a - be^v \\ v' &= -H_u(u, v) = -c + de^u. \end{aligned}$$

We can integrate each partial derivative to obtain

$$\int H_v(u, v)dv = av - be^v + C \text{ and } \int H_u(u, v)du = cu - de^u + C.$$

If we let $H(u, v)$ be the sum of the two antiderivatives above,

$$= cu + av - de^u - be^v,$$

then its immediate that

$$H_v(u, v) = a - be^v \text{ and } H_u(u, v) = c - de^u,$$

which satisfies the definition of $H(x, y)$ for a Hamiltonian system. Thus this new system using the change of variables $u = \ln(x)$, $v = \ln(y)$ is Hamiltonian using the function

$$H(u, v) = cu + av - de^u - be^v.$$

We can also write this function in terms of original variables x, y as:

$$H(x, y) = c \ln(x) + a \ln(y) - dx - by.$$

Note that we can verify the level curves $H(x, y) = C$ are closed orbits, which matches the behavior we expect based on our linearization in section 4.

In this context, we gain a few advantages from this change of variables. First, our function $H(u, v)$ is separable. By defining $F(u) = cu - de^u$ and $G(v) = av - be^v$, we can separate $H(u, v)$ into $H(u, v) = F(u) + G(v)$, a sum of a function of only u and a function of only v . Furthermore, the definition of u and v forces solutions to be positive, a property that was not enforced when applying Forward and Backward Euler to the original system. However, it is a significant property when considering biological feasibility, since populations cannot be negative in the real world.

8 Symplectic Euler

Now that we have written Lotka-Volterra as a Hamiltonian and symplectic system, we turn to symplectic solvers as a possible better choice. We introduce Symplectic Euler:

$$\begin{aligned} u_{n+1} &= u_n + h \cdot H_v(u_n, v_n) \\ v_{n+1} &= v_n - h \cdot H_u(u_{n+1}, v_n). \end{aligned}$$

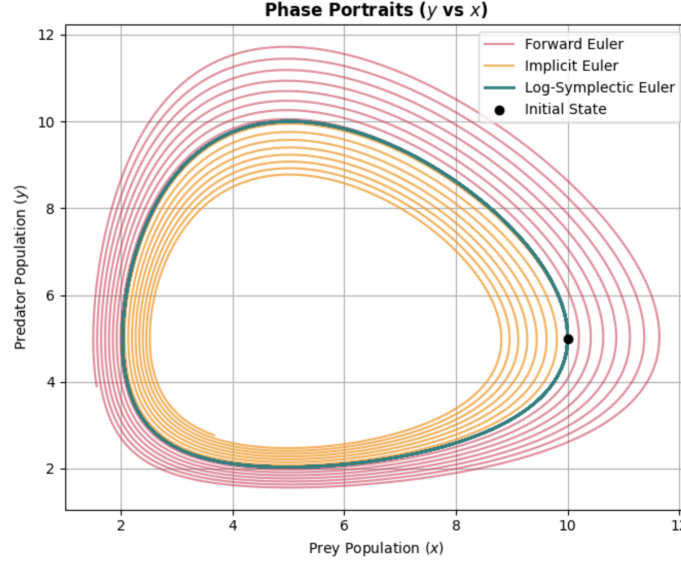
This scheme as written above, treats u as explicit, since it is defined based on solely the u_n, v_n calculated previously, and v as implicit, dependent on the current iteration's values for u_{n+1}, v_{n+1} . Here, the separability of $H(u, v)$ allows us to write the scheme explicitly, since $H_u = F'(u)$ and $H_v = G'(v)$. This is given by

$$\begin{aligned} u_{n+1} &= u_n + h \cdot (a - be^{v_n}) \\ v_{n+1} &= v_n + h \cdot (-c + de^{u_{n+1}}). \end{aligned}$$

9 Example and Results

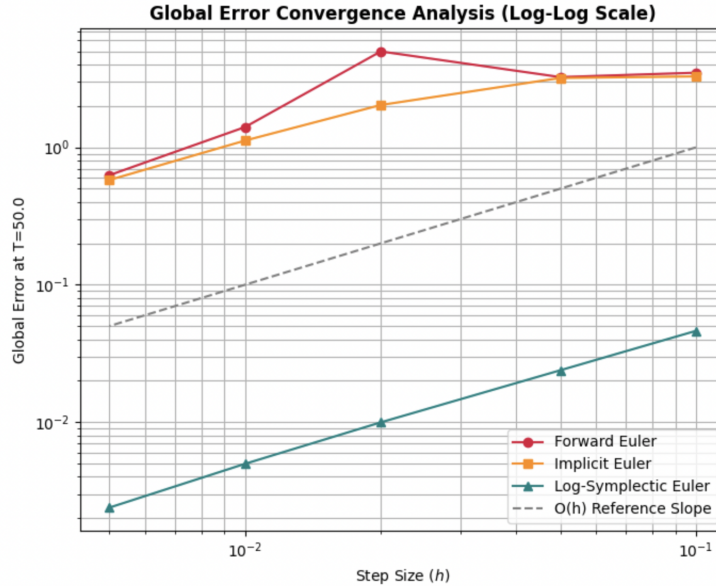
We introduce some results from a test case using the values: $a = 1, b = 0.2, c = 1, d = 0.2, t = [0, 50], h = 0.01, (x(0), y(0)) = (10, 5)$.

In this first image below, we see the phase portraits for implementing all three methods. We see Forward Euler's instability, which cause solutions to spiral outward. We see Backward Euler's stability and strict inequality causing solutions to spiral inward. But most importantly, We see that Symplectic Euler leads to a closed trajectory, exactly as desired.

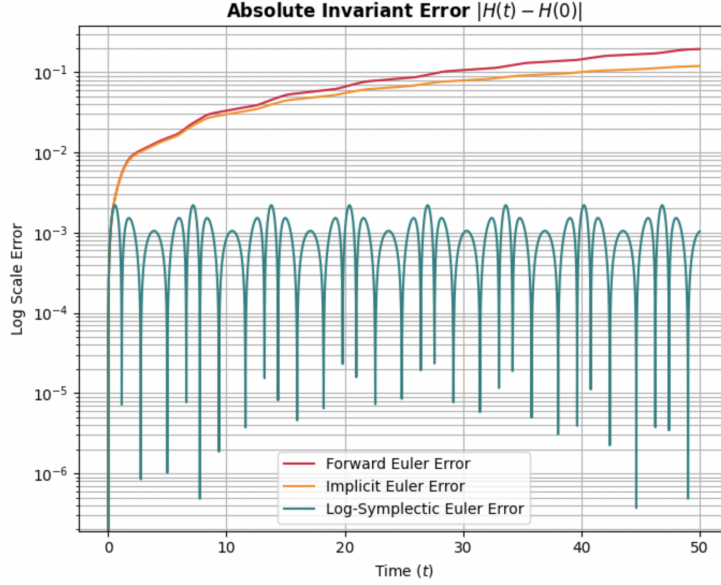


10 Error and Stability

All three solvers have local error $O(h^2)$, and thus global error $O(h)$. This property can be visualized in the graph below, which plots global error against step size h . To compute global error, we solved using Symplectic Euler with step size $h = 0.0005$ and used this as a reference solution. The dashed line is a line to represent $O(h)$ as a means of comparison.



In the next image, we see a comparison of the error of the invariant value $H(x, y) = C$. Recall that, for a perfect solution, $H(x(t), y(t))$ should be constant with time, and thus we should have $|H(x(t), y(t)) - H(x(0), y(0))| = 0$ for any point in time t . We plot the value of the error of this invariant value, $|H(x(t), y(t)) - H(x(0), y(0))|$, and observe what happens to this value with each of the three methods considered.



We see that Forward and Backward Euler both cause this value to increase greatly. This indicates to us that the solutions are not as accurate as desired, since neither Forward nor Backward Euler are able to preserve this property of the system. On the other hand, Symplectic solvers have the benefit of bounding this error. For Symplectic Euler, though the invariant error is not constant, it is clearly much more well controlled, and seems to stay within some bounded "tube". More specifically, we have that $|H(x(t), y(t)) - H(x(0), y(0))| = O(h)$, meaning a smaller step size would bound this error tighter.

For nonlinear systems, for a scheme to be locally linearly stable, we desire that, along solution paths, all eigenvalues of the Jacobian λ multiplied by timestep h satisfy $|R(h\lambda)| \leq 1$, and thus are in the linear stability domain. While this does not necessarily ensure global nonlinear stability, it gives us a suitable baseline for analyzing the behavior of these methods. For the Jacobian calculated in section 4, along most solution paths, the eigenvalues will be imaginary, since they are on closed elliptical orbits, but may have non-zero real components. It may also be helpful to consider stability local to the nontrivial equilibrium point, for which we will consider the purely imaginary eigenvalues.

The stability function for Forward Euler is given by $R(z) = 1 + z$. At the non trivial equilibrium point, the purely imaginary eigenvalues do not satisfy $|R(z)| \leq 1$. For eigenvalue $\lambda = iw$, we have $R(h\lambda) = |1 + ihw| > 1$ for any value of h . Because $\sqrt{1 + w^2 h^2} > 1$ for all choices of $h > 0$, this immediately leads us to conclude this scheme is unconditionally unstable in this context.

The stability function for Implicit Euler is $R(z) = \frac{1}{1-z}$. All imaginary eigenvalues $\lambda = u + iw$ can satisfy $|R(h\lambda)| = |\frac{1}{1-hu+ihw}| \leq 1$ for a sufficiently small step size h , making this method conditionally stable. The purely imaginary eigenvalues (like at equilibrium) always satisfy this inequality $|R(h\lambda)| = |\frac{1}{1-ihw}| < 1$ for any size of timestep, so this scheme is unconditionally stable near the equilibrium. Since we have strict inequality, solutions spiral inward near the equilibrium.

The stability region for Symplectic Euler is given by a circle centered at $(\frac{2}{h}, 0)$ with radius $\frac{2\sqrt{2}}{h}$. For this to contain the purely imaginary eigenvalues, say $\lambda = iw$, we need the timestep to satisfy $h < \frac{2}{w}$. The area-preservation property of Symplectic Euler is what ensures numerical solutions form closed orbits, maintaining stability when

applied in this context.

11 Discussion and Conclusion

Forward and Backward Euler ultimately proved to be ineffective in this context. Instead, we exploited properties of the Lotka-Volterra model to make a better choice in scheme. Employing a change of variables allowed us to view the system in a better light, and keeps solutions biologically feasible by enforcing non-negative populations. We chose to implement Symplectic Euler, which maintains the same global error with significantly better results when compared to Forward or Backward Euler. We also note that while Backward Euler has high computational cost due to the use of Newton's Method, Symplectic Euler provides superior results with a lower computational cost. We can summarize our findings in the table below:

Numerical Method:	Forward Euler	Backward Euler	Log-Symplectic Euler
Order of Global Error:	$\mathcal{O}(h)$	$\mathcal{O}(h)$	$\mathcal{O}(h)$
Computational Class:	Explicit (Fast)	Implicit (Slow, due to Newton's Method)	Explicit (Fast, due to separability of $H(x, y)$)
Population Positivity:	No	No	Yes (via variable change)
Long-Term Orbit:	Unstable Spiral Outward	Stable Spiral Inward	Closed Periodic Trajectory

Though Symplectic Euler provides a massive improvement over traditional methods for this system, it represents only a basic first-order symplectic integrator. A shortcoming of this scheme is its asymmetric treatment of the coordinates, which may introduce phase error over longer time intervals despite bounding the invariant value.

Implementing other higher-order geometric integrators can be useful to improve local accuracy while maintaining structural invariants. Specifically, the second-order Stormer-Verlet (Leapfrog) method offers a symmetric scheme that significantly narrows the error "tube" bounding the invariant value. Additionally, we could also explore Fourth-Order Symplectic Runge-Kutta algorithms.

REFERENCES

- [1] A. ARGUN, A. CALLEGARI, AND G. VOLPE, *Ecosystems*, in Simulation of Complex Systems, IOP Publishing, 2021, ch. 14, pp. 14–1–14–17, <https://doi.org/10.1088/978-0-7503-3843-1ch14>.
- [2] M. BECK AND M. J. GANDER, *On the positivity of Poisson integrators for the Lotka–Volterra equations*, BIT Numerical Mathematics, (2015), <https://www.unige.ch/~gander/Preprints/PaperBIT.pdf>. Manuscript.
- [3] A. ISERLES, *A First Course in the Numerical Analysis of Differential Equations*, Cambridge Texts in Applied Mathematics, Cambridge University Press, Cambridge, UK, second ed., 2009.
- [4] J. NIIRANEN, *Fast and accurate symmetric euler algorithm for electromechanical simulations note: The method became later known as "symplectic euler"*, 09 1999, <https://doi.org/10.13140/2.1.4663.0080>.
- [5] WIKIPEDIA CONTRIBUTORS, *Lotka–Volterra equations*, 2026, <https://wikipedia.org>. [Online; accessed 6-June-2026].
- [6] WIKIPEDIA CONTRIBUTORS, *Semi-implicit euler method — Wikipedia, the free encyclopedia*, 2026, https://en.wikipedia.org/w/index.php?title=Semi-implicit_Euler_method&oldid=1357454820. [Online; accessed 11-June-2026].
- [7] J. WYSS-GALLIFENT, *MATH 246: Chapter 3 Section 7: Hamiltonian Systems*. Lecture notes, Department of Mathematics, University of Maryland, <https://www.math.umd.edu/~immortal/MATH246/lecturenotes/ch3-7.pdf>. [Online; accessed 6-June-2026].

```

import numpy as np
import matplotlib.pyplot as plt

#parameters
a=1
b=0.2
c=1
d=0.2

#setting up system
def f(t,u):
    x,y=u[0],u[1]
    dxdt=x*a-b*x*y
    dydt=-y*c+d*x*y
    return np.array([dxdt,dydt])

#jacobian for later
def jacobian(u):
    x,y=u[0],u[1]
    return np.array([[a-b*y,-b*x],[d*y,-(c-d*x)]])

#invariant value calculation
def get_v(u_hist):
    x=u_hist[:,0]
    y=u_hist[:,1]
    return -d*x+c*np.log(x)-b*y+a*np.log(y)

#method 1: forward euler
def forward_euler(f,u0,t_span,h):
    t_vals=np.arange(t_span[0],t_span[1]+h,h)
    u=np.zeros((len(t_vals),2))
    u[0]=u0.copy()
    for i in range(len(t_vals)-1):
        u[i+1]=u[i]+h*f(t_vals[i],u[i])
    return t_vals,u

#method 2: implicit euler
def implicit_euler(f,u0,t_span,h):
    t_vals=np.arange(t_span[0],t_span[1]+h,h)
    u=np.zeros((len(t_vals),2))
    u[0]=u0.copy()
    for i in range(len(t_vals)-1):
        t_next=t_vals[i+1]
        u_curr=u[i]
        u_guess=u_curr+h*f(t_vals[i],u_curr)
        for _ in range(10):
            G=u_guess-u_curr-h*f(t_next,u_guess)
            J_G=np.eye(2)-h*jacobian(u_guess)
            delta_u=np.linalg.solve(J_G,-G)
            u_guess+=delta_u

```

```

        u[i+1]=u_guess
    return t_vals,u

#method 3: symplectic euler (using ln-transformed variables)
def symplectic_euler(u0,t_span,h):
    t_vals=np.arange(t_span[0],t_span[1]+h,h)
    u=np.zeros((len(t_vals),2))
    u[0]=u0.copy()

    #transforming initial state to log coordinates
    u_log=np.log(u0[0])
    v_log=np.log(u0[1])

    for i in range(len(t_vals)-1):
        #step 1: explicit update for u using old v
        u_log=u_log+h*(a-b*np.exp(v_log))
        #step 2: explicit update for v using new u
        v_log=v_log+h*(-c+d*np.exp(u_log))

        #transforming back to physical population values
        u[i+1]=np.array([np.exp(u_log),np.exp(v_log)])
    return t_vals,u

#conditions/timeinterval set
u0=np.array([10.0,5.0])
t_span=[0.0,50.0]
fixed_h=0.01

#implementing all 3 methods
t_fixed,u_fe=forward_euler(f,u0,t_span,fixed_h)
_,u_ie=implicit_euler(f,u0,t_span,fixed_h)
_,u_se=symplectic_euler(u0,t_span,fixed_h)

#errors
v0=d*u0[0]-c*np.log(u0[0])+b*u0[1]-a*np.log(u0[1])
v_fe=get_v(u_fe)
v_ie=get_v(u_ie)
v_se=get_v(u_se)

#using tiny step size solution as a reference for true answer
_,u_ref=symplectic_euler(u0,t_span,0.0005)
true_final=u_ref[-1]

step_sizes=[0.1,0.05,0.02,0.01,0.005]
err_fe=[]
err_ie=[]
err_se=[]

for h in step_sizes:
    _,fe_sol=forward_euler(f,u0,t_span,h)
    _,ie_sol=implicit_euler(f,u0,t_span,h)

```

```

_, se_sol=symplectic_euler(u0,t_span,h)

#calculating absolute error compared to reference solution
err_fe.append(np.linalg.norm(fe_sol[-1]-true_final))
err_ie.append(np.linalg.norm(ie_sol[-1]-true_final))
err_se.append(np.linalg.norm(se_sol[-1]-true_final))

```